

4.1.2 Programming paradigms

4.1.2.1 Programming paradigms

Content	Additional information
Understand the characteristics of the procedural- and object-oriented programming paradigms, and have experience of programming in each.	

4.1.2.2 Procedural-oriented programming

Content	Additional information
Understand the structured approach to program design and construction.	
Be able to construct and use hierarchy charts when designing programs.	
Be able to explain the advantages of the structured approach.	

4.1.2.3 Object-oriented programming

Content	Additional information
Be familiar with the concepts of: • class • object • instantiation • encapsulation • inheritance • aggregation • composition • polymorphism • overriding.	Students should know that: • a class defines methods and property/attribute fields that capture the common behaviours and characteristics of objects • objects based on a class are created using a constructor, implicit or explicit, and a reference to the object assigned to a reference variable of the class type • in the Unified Modelling Language (UML) composition is represented by a black diamond line and aggregation by a white diamond line.
Know why the object-oriented paradigm is used.	
Be aware of the following object-oriented design principles: • encapsulate what varies • favour composition over inheritance • program to interfaces, not implementation.	Students would benefit from practical experience of programming to an interface, but will not be explicitly tested on programming to interfaces or be required to program to interfaces in any practical exam.

Content	Additional information
Be able to write object-oriented programs.	Practical experience of coding for user-defined classes involving: • abstract, virtual and static methods • inheritance • aggregation • polymorphism • public, private and protected specifiers.
Be able to draw and interpret class diagrams.	Class diagrams involving single inheritance, composition (black diamond line), aggregation (white diamond line), public (+), private (-) and protected (#) specifiers.